

Plain Authentication using React + Node.js + PostgreSQL + Vercel

Complete Step-by-Step Checklist

This checklist represents the implementation **before** introducing JWT.

PART 1 - Backend (Node.js)

Step 1

Install Required Packages

Install packages required for the backend.

Examples

- Express (or Vercel Serverless Functions)
 - pg
 - bcrypt
-

Step 2

Configure PostgreSQL

Create

```
db.js
```

Responsibilities

- Import pg
- Create Pool

- Read POSTGRES_URL
 - Export Pool
-

Step 3

Configure Environment Variables

Configure

```
POSTGRES_URL
```

inside

- Vercel Environment Variables
 - Local .env (optional)
-

Step 4

Create Users Table

Example columns

- id
- name
- email
- password

Password will contain the bcrypt hash.

Step 5

Create Registration API

Create

```
register.js
```

Responsibilities

- Receive User Data
 - Validate Inputs
 - Check Duplicate Email
 - Hash Password
 - Store User in PostgreSQL
 - Return Success Response
-

Step 6

Import bcrypt

Modify

```
register.js  
login.js
```

Import bcrypt for

- Password Hashing
 - Password Comparison
-

Step 7

Hash Password During Registration

Never store plain passwords.

Workflow

```
graph TD  
  Password --> bcrypt[bcrypt.hash()]  
  bcrypt --> Hashed[Hashed Password]
```

|
▼
Save to PostgreSQL

Step 8

Create Login API

Create

```
login.js
```

Responsibilities

- Read Email
- Read Password
- Find User
- Compare Password
- Return Login Result

Step 9

Verify Password

Use

```
bcrypt.compare()
```

If comparison fails

Return

```
401 Unauthorized
```

Step 10

Return Login Response

Return

```
success  
message  
user
```

Example

```
{  
  success,  
  message,  
  user  
}
```

No JWT Token is returned.

Step 11

Handle Login Errors

Examples

```
400
```

Missing Inputs

```
401
```

Invalid Credentials

```
500
```

Internal Server Error

PART 2 - Frontend (React)

Step 12

Create Login Component

File

```
Login.jsx
```

Responsibilities

- Capture Email
- Capture Password
- Call Login API
- Read Response
- Display Success/Failure

Step 13

Receive Login Response

Backend returns

```
success  
message  
user
```

Step 14

Store User Information (Optional)

Store

```
user
```

inside

```
localStorage
```

Example

- id
- name
- email

No Token exists.

Step 15

Display User Information

Examples

```
Welcome Alex
```

Display

- Name
- Email

using stored user information.

Step 16

Logout

Remove

```
user
```

from Local Storage.

PART 3 - API Requests

Step 17

Call APIs

React directly calls APIs.

Example

```
GET /employees
```

No Authentication Header.

No Authorization Header.

Step 18

Backend Executes Request

Backend immediately processes

```
employees.js
```

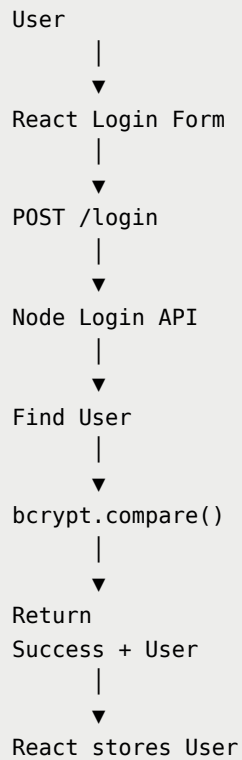
No identity verification.

No token verification.

PART 4 - Login Flow

Step 19

Authentication Flow



PART 5 - Protected Pages

Step 20

Protect Pages (Client Side Only)

React checks

localStorage

Example

User Exists

Allow page.

Otherwise

Redirect to Login.

This is only a UI convenience and is **not secure**.

PART 6 - Files

Backend

Existing Files

- package.json
- db.js
- register.js
- login.js
- cors.js
- vercel.json

Environment

- POSTGRES_URL
-

Frontend

Existing Files

- Login.jsx
- config.js

Optional

- Welcome.jsx
 - Logout.jsx
-

Packages Used

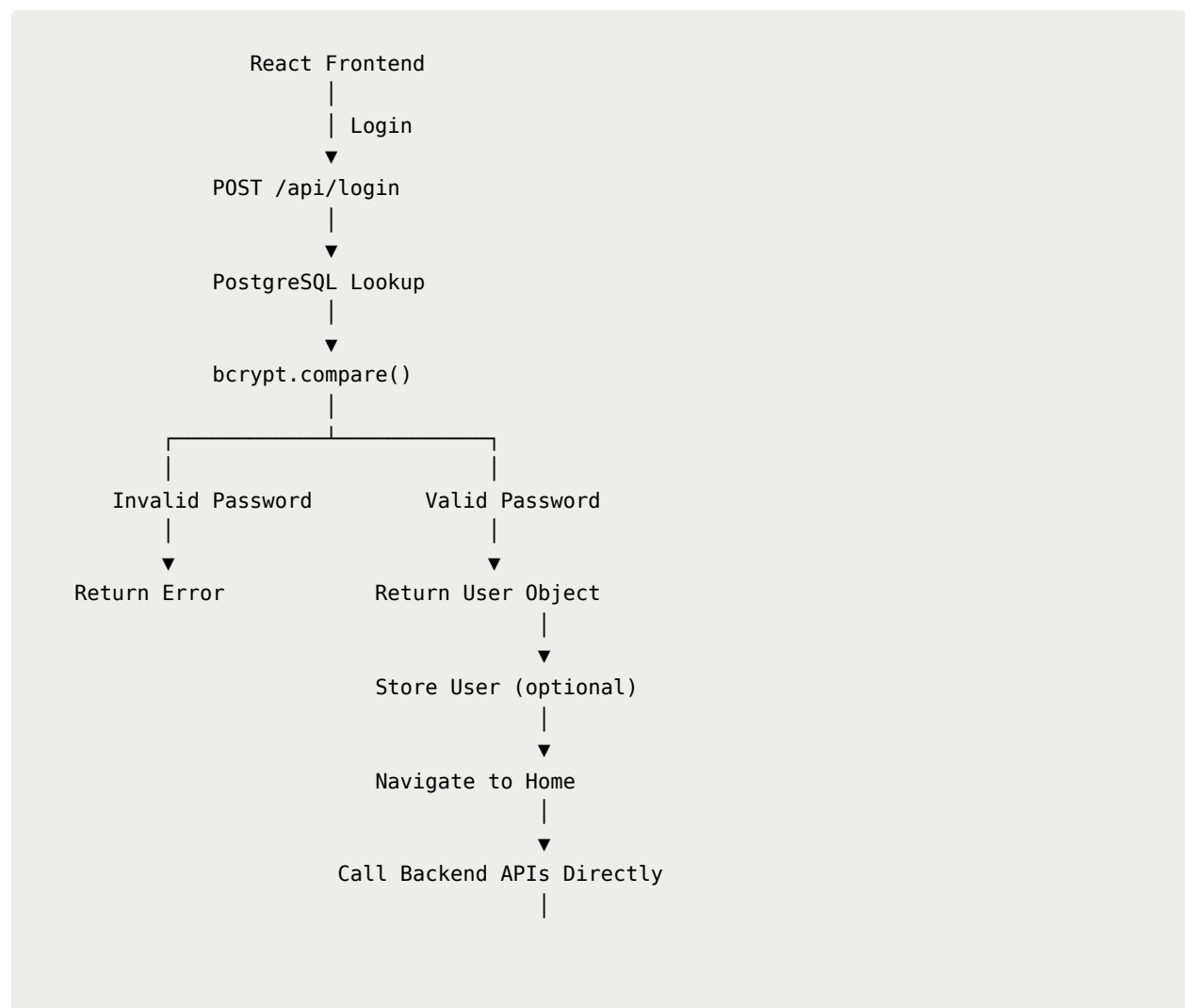
Backend

- pg
- bcrypt

Frontend

No authentication packages required.

Plain Authentication Architecture



Backend Executes Request

Plain Authentication vs JWT Authentication

Feature	Plain Authentication	JWT Authentication
Password hashing	✓ bcrypt	✓ bcrypt
User lookup	✓ PostgreSQL	✓ PostgreSQL
Login verification	✓ bcrypt.compare()	✓ bcrypt.compare()
Token generated	✗ No	✓ JWT
JWT Secret	✗ Not Required	✓ Required
Token expiry	✗ Not Available	✓ Supported
Authorization header	✗ No	✓ Bearer <token>
Protected APIs	✗ No	✓ Yes
User identity on every request	✗ Not Verified	✓ Verified using JWT
Suitable for production	✗ No	✓ Yes